Kubernetes · Platform Engineering · Automation · Reliability

# 포트폴리오

김지효
z33hyo@gmail.com
github.com/z-kim

## 프로젝트

kt cloud 클라우드 인프라 부트캠프 2기에서 수행한 네 개의 팀 프로젝트 중, 제가 담당한 구현과 설계 판단을 중심으로 정리했습니다.


| | 포트폴리오 항목 | 주요 담당 내용 | 페이지 |
|---|---|---|---|
| 01 | **멀티클라우드 DR 운영·검증 자동화**<br>팀 프로젝트: 멀티클라우드 이커머스 DR | 인프라 생성·삭제 및 DR 전환 자동화<br>접수 주문 정합성 검증 | 2 |
| 02 | **K3s·Argo CD GitOps 운영 구성**<br>팀 프로젝트: 클라우드 네이티브 통합 보안 플랫폼 | GitOps 배포·저장소 인증·워크로드 설정,<br>Kafka 기반 보안 이벤트 수집 파이프라인 구축 | 8 |
| 03 | **PR Preview 환경 생성·정리 자동화**<br>팀 프로젝트: PR Preview 플랫폼 | PR별 격리 환경 생성·배포,<br>PR 종료·TTL 만료 시 자동 정리 | 11 |
| 04 | **AWS 인프라 검증·장애·부하 테스트**<br>팀 프로젝트: IaC 기반 클라우드 인프라 자동화 | 인프라 상태·장애 복구·확장 검증,<br>외부 요청의 성공률·응답 성능 측정 | 13 |


## 기술 스택

| 영역 | 기술 | 적용 | 프로젝트 |
|---|---|---|---|
| Kubernetes | K3s, EKS, AKS, Helm, HPA, KEDA | 클러스터 구성·워크로드<br>배포·오토스케일링 | 01 02 03 |
| GitOps | Argo CD, Kustomize | Sync Wave 배포 순서 제어,<br>Overlay 기반 DR 상태 관리 | 01 02 |
| CI/CD · 자동화 | GitHub Actions, GitHub App,<br>Docker, Trivy | 빌드·배포·인프라 생성·DR<br>전환 자동화, 권한 분리 | 01 02 03 |
| 클라우드 | AWS (VPC, ALB, ASG, EKS, RDS,<br>SSM, VPN), Azure (VNet, AKS,<br>PostgreSQL, Key Vault), Cloudflare | 클라우드 환경 생성·운영<br>자동화, 서비스 배포·장애 검증 | 01 02<br>03 04 |
| IaC · 구성관리 | Terraform, Ansible | 스택 실행 순서·Output 연계,<br>K3s 노드 설정,<br>AWS 테스트 환경 구성 | 01 02 04 |
| 모니터링 | Prometheus, Grafana | 메트릭 기반 상태 확인,<br>모니터링 구성 연계 | 01 02 04 |
| 테스트 · 검증 | pytest, Locust, k6 | 인프라 상태 검증, 장애·부하<br>테스트 | 01 04 |
| 보안 · 인증 | External Secrets, IAM Role, Falco,<br>ModSecurity | Secret 동기화·역할 기반<br>인증, 보안 이벤트 수집 | 01 02 |
| 데이터 · 메시징 | PostgreSQL, MySQL, Kafka | DB 복제·Writer 전환,<br>Kafka Replay·이벤트 수집,<br>PR별 DB 관리 | 01 02 03 |
| OS · 언어 | Linux, Python | 서버 환경 구성·운영, 자동화·<br>검증 스크립트 작성 | 01 02<br>03 04 |

# 01 멀티클라우드 DR 운영·검증 자동화

팀 프로젝트 · 멀티클라우드 이커머스 DR

**2026.04.16 – 07.21 · 7명**

https://github.com/z-kim/ktcloud-multi-cloud-dr

오픈소스 이커머스 데모인 Online Boutique를 기반으로 AWS Active–Azure Warm Standby DR 환경을 구축했습니다. AWS 장애가 발생해도 접수 완료된 주문이 유실되지 않고 주문 처리를 재개할 수 있도록 PostgreSQL logical replication으로 DB 상태를 Azure에 복제하고, 장애 전환 시 Kafka Replay로 미처리 주문 이벤트를 재처리하도록 구성했습니다.

저는 GitOps·DR Workflow를 담당해 인프라 생성·삭제와 DR 전환·복귀 절차를 자동화하고, 주문 수신 재개와 접수 완료 주문의 유실·중복 여부를 확인하는 검증 절차를 구현했습니다.

## 완료 기준

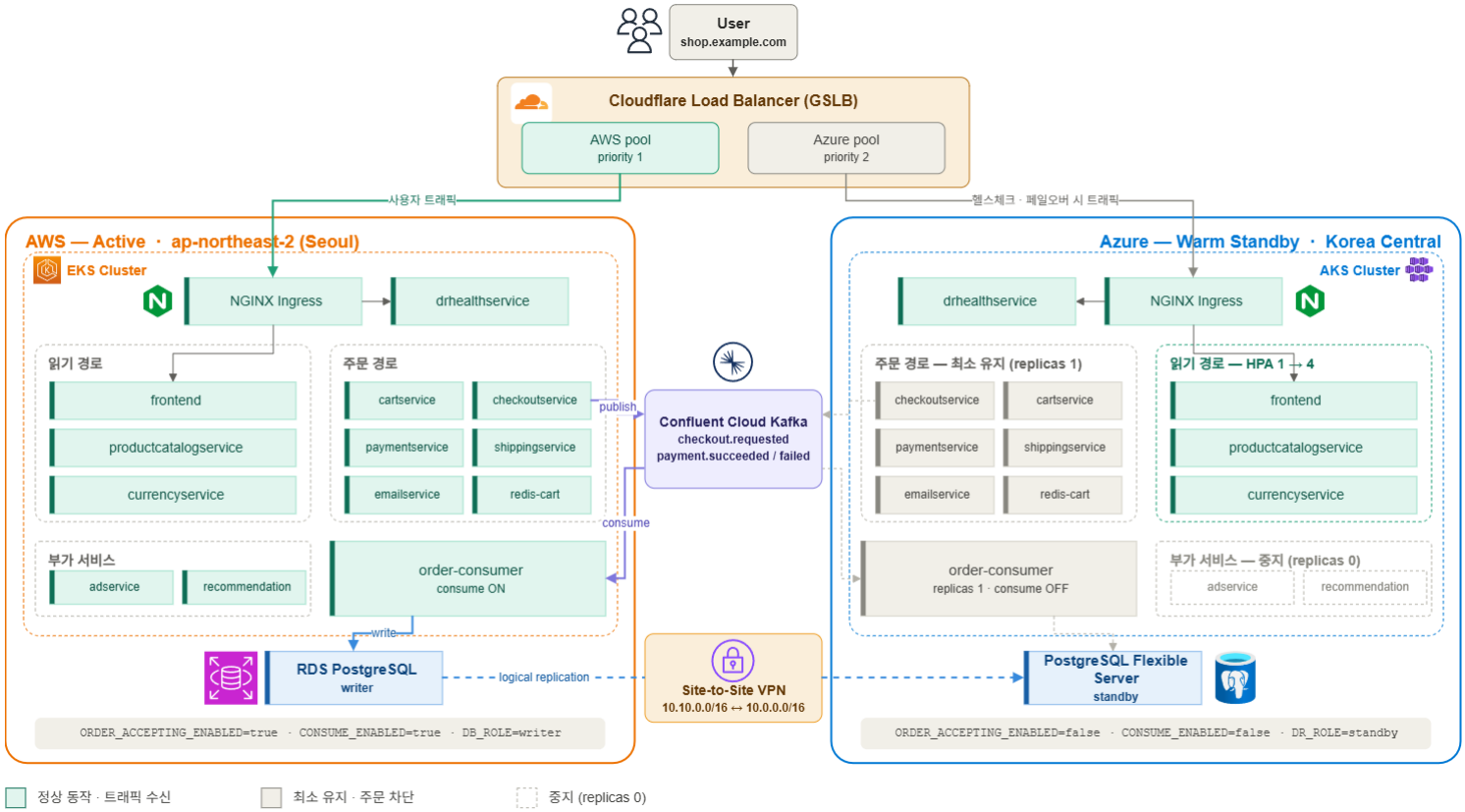
- AWS 장애 시 RTO 15분 이내
- 접수 완료 주문 기준 RPO 0(유실 0건) · 중복 0건

**결과 요약** Failover 주문 불가 497초 · 접수 주문 유실 0건 · 중복 0건

## 담당 범위

7명 팀은 AWS 인프라 / Azure 인프라 / Cloudflare GSLB·장애 감지·트래픽 전환 / Kafka / PostgreSQL·Replication / GitOps·DR Workflow / PM으로 역할을 나눴습니다. 저는 GitOps·DR Workflow를 담당했습니다. 각 영역에서 필요한 입력값과 실행 순서를 정리하고, Deploy/Destroy, Failover/Failback, DR 검증 절차를 각각의 Workflow로 구현했습니다. 담당 범위는 다음과 같습니다.

- 인프라 생성·삭제 워크플로우 (Deploy All / Destroy All)
- GitOps 운영 상태 설계와 Overlay 구현
- DR Failover / Failback 자동화
- 정상·장애·DR 검증 자동화와 Drill 리포트
- 애플리케이션 부분 장애 격리




멀티클라우드 DR 서비스 구성도

## 01-1 Warm Standby 설계

RTO 15분 요구사항과 상시 운영 비용을 고려해 Active-Active 대신 Azure Warm Standby를 선택했습니다. Azure에는 상품 조회부터 주문 완료까지 필요한 서비스만 상시 유지하고, 광고·추천 등 주문과 무관한 부가 서비스는 replica 0으로 설정했습니다.

checkout·payment·shipping·email·cart는 최소 replica로 유지했습니다. 상품 조회 요청을 처리하는 frontend·productcatalogservice·currencyservice는 평상시 replica 1로 유지하고, 전환 시에는 HPA로 최대 4개까지 확장되도록 구성했습니다.

## 01-2 인프라 수명주기 자동화

프로젝트 실습 비용을 줄이기 위해 개발 환경을 매일 생성·삭제했고, 여러 Terraform 스택의 실행 순서와 스택 간 전달값을 반복해서 관리해야 했습니다. 생성·삭제 절차를 각각 하나의 Workflow로 묶어, 같은 순서로 반복 실행할 수 있도록 했습니다.

**Deploy All**은 Terraform 스택 실행, Runtime Secret 저장, GitOps 설정 반영, Argo CD bootstrap, DB 운영 준비를 순서대로 수행하도록 구성했습니다. **Destroy All**은 리소스 간 의존성과 외부 연결을 고려한 순서로 환경을 정리하도록 구성했습니다.

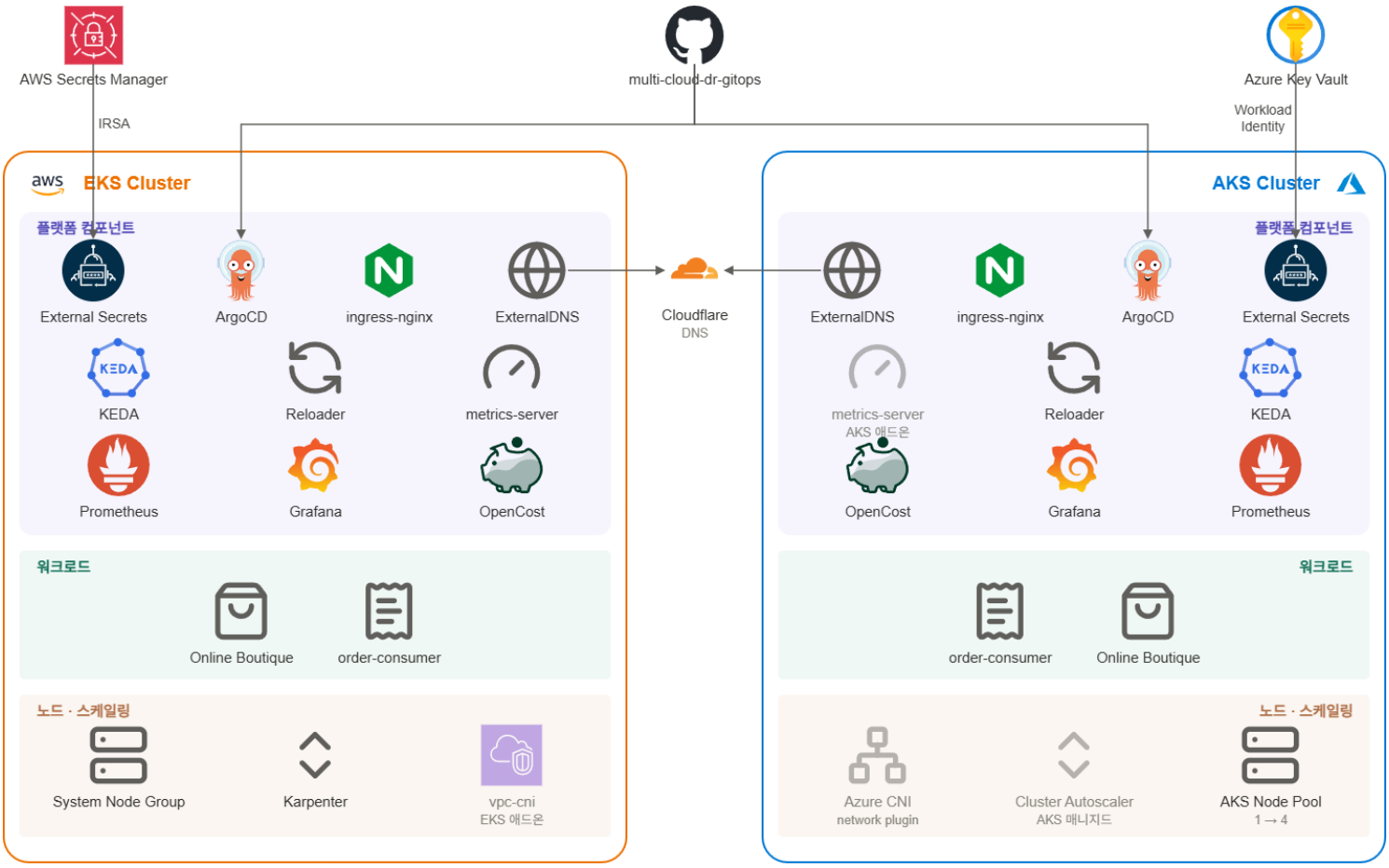
- AWS VPC/EKS/RDS, Azure VNet/AKS/PostgreSQL, Confluent Kafka, Cloudflare 스택 실행 순서 정의
- AWS–Azure site-to-site VPN과 PostgreSQL replication 절차를 구축·정리 워크플로에 연결
- Terraform output을 실행 환경 설정과 GitOps 운영 값으로 전달 (DB host, Kafka bootstrap, ExternalSecret 참조)
- AWS Secrets Manager · Azure Key Vault와 ExternalSecret 연계

**반복 실행을 통한 Workflow 개선.** Deploy All/Destroy All을 반복 실행하면서 확인한 여러 오류를 해결하고, 작업 순서·의존관계·리소스 삭제 완료 대기 조건을 보완했습니다.

대표적으로 Destroy All에서는 EKS 삭제 후 남은 VPC CNI ENI와 cluster security group 때문에 VPC 삭제가 실패해 잔여 리소스 정리 단계를 추가했습니다. Deploy All에서는 DB replication 설정 실패로 Cloudflare GSLB 구성 작업까지 중단되지 않도록, 두 작업의 실행 의존성을 분리했습니다.

## 01-3 EKS·AKS 런타임 구성

Argo CD, External Secrets 등 공통 플랫폼 구성요소는 EKS와 AKS 양쪽에 배치하고, 인증·Secret 저장소·노드 확장처럼 클라우드별로 다른 부분은 각 환경에 맞게 분리했습니다. GitOps 저장소는 하나로 두고 EKS·AKS 경로를 나눠 관리했습니다.




흐리게 표시된 항목은 클라우드가 매니지드로 제공합니다.

EKS · AKS 런타임 구성요소

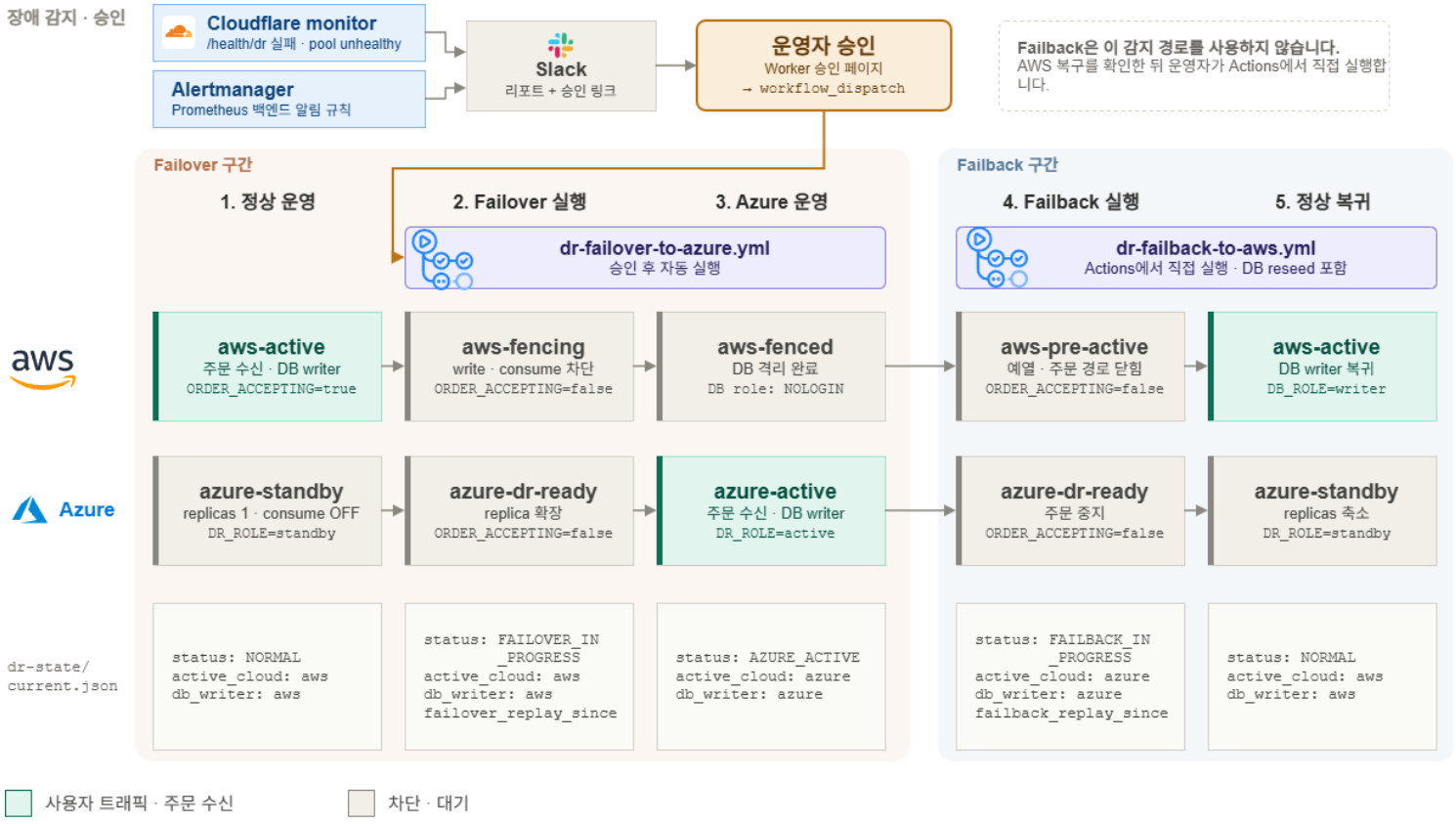
- Argo CD와 External Secrets를 양쪽에 배치하고, Secret은 클라우드별 저장소(Secrets Manager / Key Vault)에서 IRSA·Workload Identity로 참조
- ingress-nginx와 ExternalDNS로 진입 경로를 구성하고 Cloudflare DNS에 등록
- HPA는 Metrics Server에서 수집한 CPU 지표, KEDA는 Kafka lag를 기준으로 동작하도록 구성
- Secret·ConfigMap 변경 시 Reloader가 워크로드를 갱신
- 노드 확장은 EKS에서 Karpenter, AKS에서 Cluster Autoscaler로 처리
- Prometheus·Grafana·OpenCost로 상태와 비용을 관측

## 01-4 DR 운영 상태와 전환 안전성

DR 전환 과정을 Active/Standby 두 상태만으로 구분하기는 어려웠습니다. Azure의 서비스 준비 완료와 주문 처리 활성화 시점을 분리하고, 주문 처리는 DB Writer 전환과 Replay가 끝난 뒤에만 활성화했습니다. 주문 수신·Consumer 실행·Scaling 설정은 운영 상태에 따라 7개 Overlay로 관리했습니다.

| | 상태 | 내용 |
|---|---|---|
| **AWS** | `aws-active` | 주문 수신 · DB writer |
| | `aws-fencing` | write·consume 차단 진행 |
| | `aws-fenced` | DB role NOLOGIN · 격리 완료 |
| | `aws-pre-active` | 복귀 준비 · 주문 경로는 닫힘 |
| **Azure** | `azure-standby` | 최소 replica · consume OFF |
| | `azure-dr-ready` | replica 확장 · 주문 수신 전 |
| | `azure-active` | 주문 수신 · DB writer |

DR Workflow에서는 Argo CD Application의 배포 경로를 변경해 각 클러스터에 DR 상태별 Overlay를 적용했습니다.




DR 상태 전환 다이어그램

### 클라우드별 active-core 재사용

`aws-pre-active`와 `azure-dr-ready`는 각 클라우드의 active 상태와 같은 replica·HPA 구성을 사용하되 주문 처리는 비활성화했습니다. 각 클라우드에서 공통으로 사용하는 replica·HPA 설정은 `active-core`로 묶고, 주문 수신 flag와 Kafka lag 기반 KEDA 등 상태별 차이만 최종 Overlay에서 제어했습니다.

### 전환 상태 기록과 중복 실행 방지

Failover/Failback이 여러 Workflow로 나뉘어 있어 다음 실행에서 현재 DR 상태를 판단할 기준이 필요했습니다. `dr-state/current.json`에 현재 전환 상태, Active Cloud, DB Writer, 진행 중인 incident와

Replay 기준 시각을 기록했습니다. 각 Workflow는 시작할 때 이 상태를 확인해 중복 실행과 잘못된 방향의 전환을 차단했습니다.

### 트래픽 자동 전환과 DR 운영자 승인

Cloudflare Monitor는 `/health/dr`을 통해 `frontend` 상태를 확인하고, Load Balancer는 AWS Pool 장애 시 Azure Pool로 트래픽을 자동 전환하도록 구성되었습니다.

backend 장애 시에는 Alertmanager 알림을 바탕으로 Slack에 장애 보고서를 전달하고, 운영자가 DR 전환 여부를 판단하도록 했습니다. AWS fencing·DB Writer 전환·Kafka Replay 등 Failover 절차는 운영자 승인 후 실행했습니다. Failback은 자동으로 실행하지 않고, 운영자가 AWS 복구 상태를 확인한 뒤 별도 Workflow로 실행하도록 했습니다.

Slack으로 전달되는 장애 보고서에 Failover 승인 URL을 포함하고, 장애 정보와 Replay 기준 시각을 함께 전달해 승인 후 필요한 값을 다시 입력하지 않도록 했습니다.

### 동시 DB Write 방지를 위한 이중 차단

DR 전환 중 AWS와 Azure에서 동시에 주문을 DB에 기록하지 않도록 애플리케이션과 DB 양쪽에서 Write를 차단했습니다. 애플리케이션에서는 신규 주문 접수와 Consumer의 메시지 소비를 중단했고, DB에서는 Writer 계정에 `NOLOGIN`을 적용하고 기존 Writer 세션을 종료했습니다.

## 01-5 운영 Workflow와 권한 분리

Failover/Failback을 실행할 때 검증용 장애 주입이나 부하 생성이 함께 실행되지 않도록 운영 Workflow와 DR Drill을 분리했습니다. 운영 Workflow에는 GitOps 상태 전환·DB Writer 제어· Kafka Replay· Cloudflare 복구 등 전환·복구에 필요한 작업을 두고, DR Drill에서는 장애 주입· 트래픽 생성·전환 결과 검증·리포트 생성을 진행하도록 구성했습니다.

주문 처리·DB 운영·Writer 전환 작업은 서로 다른 DB 계정과 Kubernetes Namespace로 분리했습니다.

### GitHub App 기반 권한 분리

GitOps 저장소 변경과 DR 상태 기록, 애플리케이션 저장소 조회, Cloudflare Worker의 DR Workflow 실행 요청을 각각 별도의 GitHub App으로 분리했습니다. 각 App에는 해당 작업에 필요한 최소 권한만 부여했습니다.

## 01-6 애플리케이션 부분 장애 격리

장애 검증 과정에서 currencyservice 장애가 상품 목록·상세·장바구니의 500 오류로 이어지는 문제를 확인했습니다. 이에 따라 다른 서비스의 장애가 화면 전체로 전파되지 않도록 하고, 기능별로 실패 상황을 구분해 처리했습니다.

- **환율 변환** — 실패 시 기준 통화로 표시해 화면 유지
- **장바구니 조회** — 상품 목록·상세에서는 조회가 실패해도 개수만 비우고 화면을 유지하되, 장바구니 화면에서는 오류로 처리
- **주문 실패** — 접수 비활성화·주문 준비 실패·이벤트 발행 실패· 결제 실패를 구분해 안내 문구와 상태 코드를 다르게 적용
- **오류 안내** — 내부 오류 문자열 대신 사용자 안내 문구와 request id를 표시하고, 상세 정보 노출 여부는 환경 변수로 제어

## 01-7 DR 검증과 실행 결과

DR 전환 후에는 Kubernetes 상태, 주문 처리, 데이터 보존을 각각 확인해야 했습니다. 이 항목들을 반복해서 확인할 수 있도록 Runtime, 주문 E2E, DB 정합성, Autoscaling, 장애 알림, Failover/Failback 시나리오를 자동화했습니다.

### 검증 범위

| | |
|---|---|
| **런타임·준비 상태** | Deployment·Pod Ready, Replica 수, Argo CD Sync/Health, ExternalSecret 동기화, Ingress 외부 접근 |
| **주문 경로·데이터** | checkoutservice → Kafka → order-consumer → PostgreSQL 전 구간 확인, 주문 중복 저장 대조 |
| **확장·장애 대응** | HPA·KEDA 스케일 동작, checkoutservice·Kafka·DB 장애 알림, 라우팅 전환·복구 |
| **DR 실행·리포트** | Failover·Failback 실행, RTO·RPO 및 Accepted 주문 유실·중복 검증 |

DR Drill의 장애 주입, 알림, Traffic Driver, Failover/Failback, Cleanup, Summary는 각각 별도의 Workflow로 분리했습니다. 모든 Workflow에 같은 `incident_id`를 전달해 summary와 artifact를 Drill 단위로 묶었습니다.

### 접수 완료 주문의 유실·중복 검증

k6 기반 Traffic Driver가 주문 결과를 접수·거절·실패로 구분해 기록하고, 접수 성공 응답을 받은 테스트 주문의 식별값을 별도로 저장했습니다. Failover/Failback 이후 해당 식별값을 기준으로 접수 기록과 PostgreSQL의 주문 기록을 대조해 접수 완료 주문의 유실·중복 여부를 확인했습니다.

### DR Drill 실행 결과

| 항목 | 결과 |
|---|---|
| Drill 결과 | PASS |
| 부하 조건 | failover/failback 각 20분, VUs 4, order probability 0.45 |
| 주문 불가 구간 | failover 497초 / failback 703초 |
| 주문 시도 / 접수 / 거절 / 실패 | 3,329 / 1,392 / 1,520 / 417 |
| 접수 주문 유실 / 중복 | 0 / 0 |
| RPO | 0 (접수 완료 주문 기준) |

Failover에서 장애 발생 후 주문 수신이 재개되기까지 497초가 걸려 목표 RTO 15분을 충족했습니다.

그 밖에도 다음 항목을 검증했습니다.

- Normal order E2E — 시도 61건, 접수 61건, 실패 0
- HPA — frontend 2 → 4
- KEDA — order-consumer 2 → 4, `checkout.requested` 1,200건 burst
- Backend Alert Drill — 장애 주입 시 Alertmanager 알림, 장애 해제 후 정상화 확인

## 01-8 한계와 후속 개선안

**Failover 대상 장애 구분.** Kafka처럼 두 클라우드가 공통으로 의존하는 서비스에 장애가 발생하면, 클라우드 전환만으로는 복구할 수 없습니다. 후속 개선안으로 Precheck에 공통 의존성의 상태 점검 등을 추가하고, 장애 유형에 따라 Failover가 적절한 대응인지 판단하는 방안을 검토했습니다.

**주문 수신 재개 시점.** Cloudflare 라우팅 설정은 Failover 시작 후 25초 만에 Azure로 변경됐지만, 외부에서 측정한 주문 불가 시간은 497초였습니다. 기존에는 DB Writer 전환과 Kafka Replay를 완료한 뒤 주문 수신을 재개했습니다. 후속 개선안으로는 DB Writer 전환과 주문 경로 확인 후 주문 수신을 먼저 재개하고, Kafka Replay와 정합성 검증은 이후 수행해 주문 접수 기준 RTO를 줄이는 방안을 검토했습니다.

**여러 Workflow의 실행 상태 통합.** DR Drill을 여러 GitHub Actions Run으로 나눠 실행하면서 진행 상황과 결과를 한곳에서 확인하기 어려웠습니다. 실행 순서·승인·진행 상태와 artifact·summary를 통합 관리하는 별도 운영 도구를 검토했습니다.

# 02 K3s·Argo CD GitOps 운영 구성

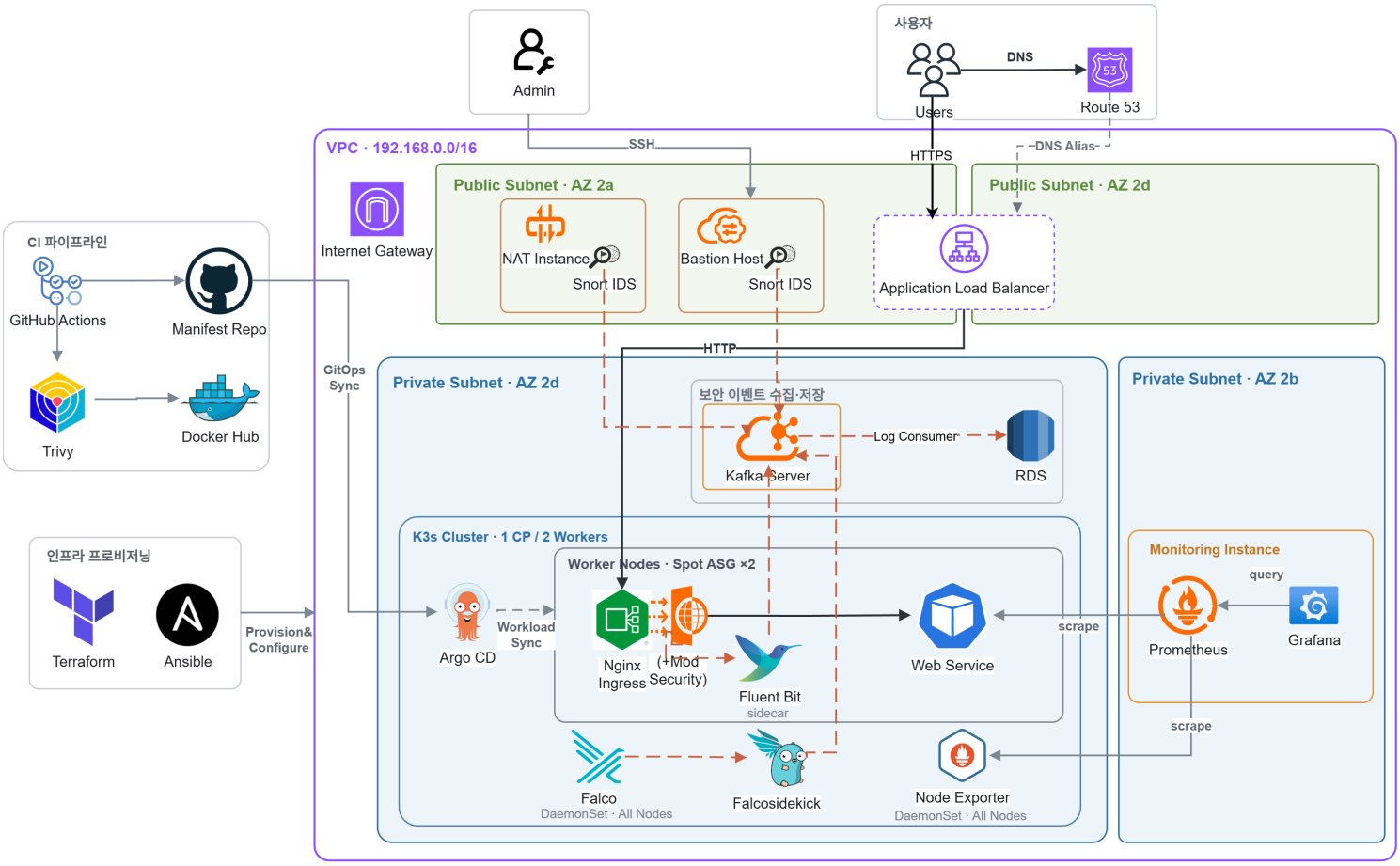
팀 프로젝트 · 클라우드 네이티브 통합 보안 플랫폼

**2026.03.10 – 04.09 · 6명**

https://github.com/z-kim/ktcloud-sixsense-k3s-cluster

Snort·Falco·ModSecurity로 네트워크·런타임·HTTP 요청 계층의 보안 이벤트를 탐지하고, Trivy로 컨테이너 이미지 취약점을 검사하는 K3s 기반 통합 보안 플랫폼을 구축했습니다.

저는 K3s 실행 환경과 Argo CD 기반 GitOps 배포, Kubernetes 워크로드 설정을 담당했습니다. 또한 ModSecurity 감사 로그와 Falco 런타임 이벤트를 Kafka로 전달하는 수집 경로를 구현했습니다.




K3s · GitOps · 보안 이벤트 수집 아키텍처

### 02-1 Worker 등록 · 노드 통신 문제 해결

초기 K3s 구성에서 Worker 등록과 노드 간 통신 문제가 있어 관련 설정을 보완했습니다. Worker ASG, k3s_token, control-plane Ansible 설정과 노드 통신 Security Group을 조정해 1 Control Plane / 2 Worker 구성을 확인했습니다.

### 03-2 GitOps 전환과 적용 순서 설계

초기에는 kubectl 기반 스크립트로 리소스를 배포했습니다. 이후 배포 설정의 변경 이력을 Git에서 관리하고, 클러스터에 실제로 적용된 상태를 쉽게 확인할 수 있도록 Argo CD App-of-Apps로 전환했습니다. 구성요소를 Application 단위로 나누고, 의존관계에 따라 Sync Wave를 지정해 구성요소를 추가하거나 변경할 때도 배포 순서를 관리할 수 있도록 했습니다.

**Application 단위 — Sync Wave.** 외부 진입 경로와 기본 관측 구성을 먼저 적용하고, 런타임 보안 구성 이후 애플리케이션을 적용하도록 순서를 나눴습니다.

| Wave | Application | 배치 기준 |
|---|---|---|
| 1 | ingress-nginx, node-exporter | 외부 진입 경로 · 기본 관측 구성 먼저 적용 |
| 2 | falco | 애플리케이션 배포 전 런타임 보안 구성 적용 |
| 3 | doc-converter | 공통 인프라 · 보안 구성 이후 앱 적용 |

**리소스 단위 — HelmChartConfig와 HelmChart.** HelmChart가 HelmChartConfig보다 먼저 적용되면 보안 · 로그 설정이 반영되지 않은 기본값으로 설치될 수 있습니다. HelmChartConfig에 sync-wave: -1, HelmChart에 0을 지정해 HelmChartConfig가 먼저 적용되도록 했습니다.

**GitOps 초기 구성 — Bootstrap.** Sync Wave는 Argo CD의 동기화 과정에서 리소스의 적용 순서를 지정하므로, Argo CD 설치와 GitOps 배포에 필요한 초기 구성은 별도의 Bootstrap 절차로 분리했습니다. Namespace · Repository Secret · Kafka 연결 설정 등을 준비한 뒤 root-app을 적용하도록 했습니다.

### 02-3 GitOps 저장소 인증과 AWS 접근 권한 분리

Private Repository 인증 방식으로 SSH Deploy Key와 GitHub App을 비교했습니다. 단기 토큰과 권한 관리 · 확장성을 고려해 읽기 전용 GitHub App을 선택했습니다. 인증 정보는 AWS SSM Parameter Store에 저장하고, External Secrets를 통해 Argo CD Repository Secret으로 주입했습니다.

AWS 접근에는 장기 Access Key 대신 노드 IAM Role을 사용했습니다. control-plane Role은 Argo CD 자격정보 조회에, worker Role은 애플리케이션의 S3 접근에 사용했습니다.

### 02-4 보안 이벤트 수집 경로

ModSecurity 감사 로그와 Falco 런타임 이벤트가 Kafka로 전달되도록 각각의 수집 경로를 구성했습니다. 수집 설정은 실제 Kafka Server 주소 대신 클러스터 내부의 kafka.logging을 참조하도록 했습니다.

| | |
|---|---|
| ModSecurity | Ingress 요청 → 검사 → audit log → fluent-bit sidecar → kafka.logging → Kafka Server |
| Falco | 노드 런타임 이벤트 → Falco → Falcosidekick → kafka.logging → Kafka Server |

실제 Kafka Server 주소는 Service와 EndpointSlice에서 관리해, 주소가 바뀌어도 ModSecurity와 Falco의 수집 설정은 수정하지 않도록 했습니다.

### 02-5 환경별 설정 분리와 배포 안정성

공통 Manifest는 base로 관리하고, 환경별 리소스 설정과 이미지 태그는 overlay로 분리했습니다.

Pod가 특정 노드에 집중되지 않도록 anti-affinity · topology spread를 적용하고, startup · readiness · liveness probe로 기동 상태와 요청 처리 준비 여부 등을 확인하도록 했습니다. Rolling Update는 배포 중 가용성을 유지하도록 `maxUnavailable: 0` / `maxSurge: 1`로 설정했습니다.

배포 이미지의 버전을 Git 커밋과 연결하기 위해 `github.sha`로 태깅하고, Kustomize newTag를 갱신했습니다. Trivy에서 `CRITICAL` 취약점이 발견되면 배포 Workflow를 중단하도록 했습니다.

### 02-6 실행 결과

- **클러스터** — K3s 1 Control Plane / 2 Worker 구성, Worker 등록 및 노드 간 통신 확인
- **GitOps · 배포** — Bootstrap 이후 Argo CD Sync Wave에 따른 Application 적용 순서 확인, doc-converter에 Pod 분산 · Rolling Update 동작 확인
- **인증 · 이벤트** — GitHub App 기반 Private Repository 연동 확인, ModSecurity · Falco 이벤트의 Kafka 전달 확인

# 03 PR Preview 환경 생성·정리 자동화

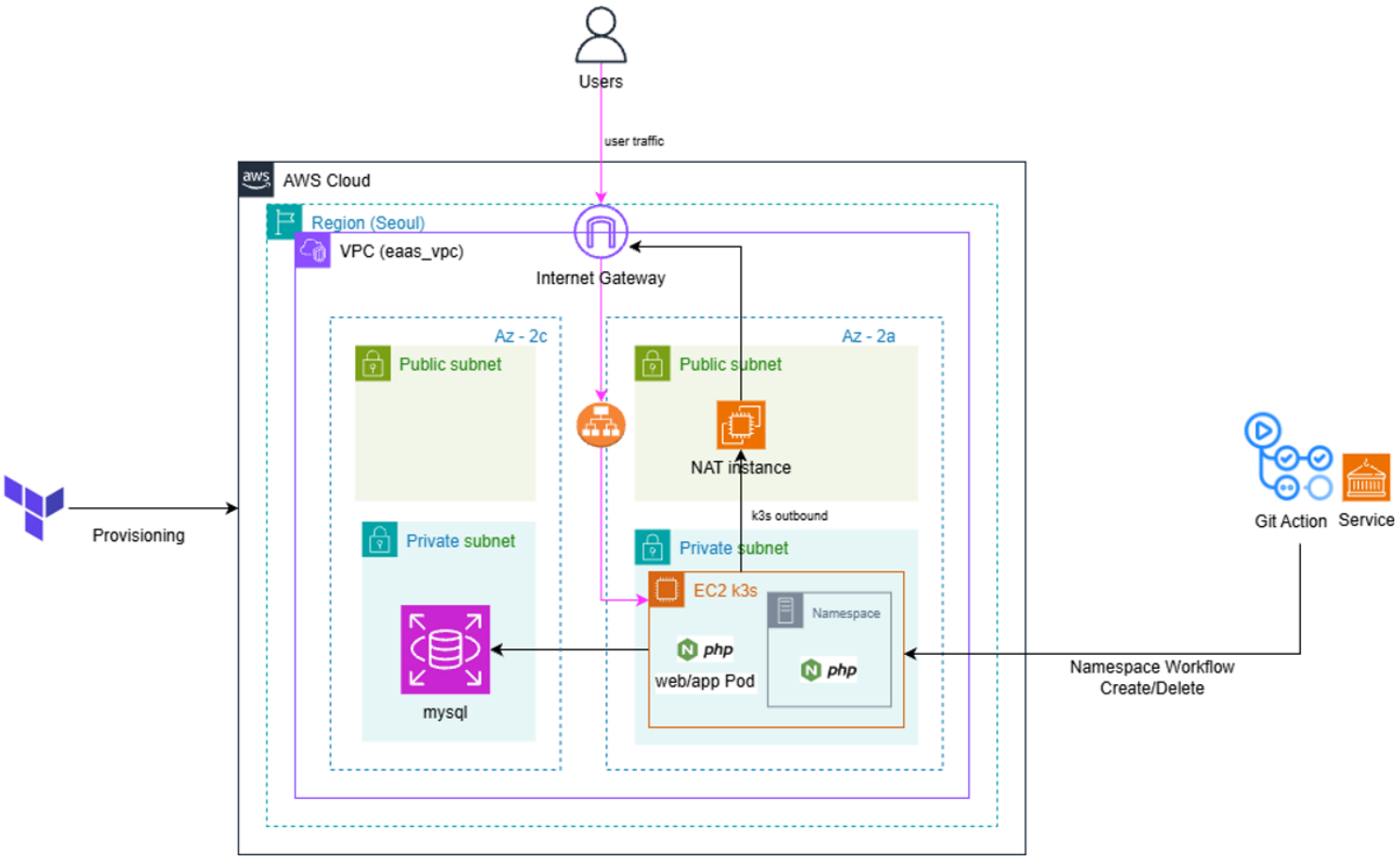
팀 프로젝트 · PR Preview 플랫폼

**2026.04.13 – 04.15 · 4명 · 3일 해커톤**

https://github.com/z-kim/ktcloud-hackaton-a-cicd

리뷰어가 PR 변경사항을 별도 배포 작업 없이 확인할 수 있도록, PR마다 격리된 Preview 환경을 자동 생성하는 플랫폼을 구축했습니다.

저는 GitHub Actions로 PR별 애플리케이션 배포와 MySQL DB 생성을 자동화했습니다. 또한 PR에 새 커밋이 추가되면 기존 배포를 갱신하고, PR 종료 또는 TTL 만료 시 환경을 정리하도록 구성했습니다.




PR Preview 생성·격리·정리 구성

## 03-1 PR 번호 기반 환경 격리

Namespace, DB, Host, image tag에 모두 PR 번호를 사용했습니다.

| | |
|---|---|
| Namespace | `preview-pr-<번호>` |
| Database | `pr_<번호>` |
| Host | `pr-<번호>.<base-domain>` |
| Image tag | `pr-<번호>-<short sha>` |

동일 PR에 새 커밋이 추가될 때는 기존 환경을 갱신하고, PR 번호를 Namespace·DB·Host의 공통 식별자로 유지했습니다. PR 번호와 기본 도메인 변수 `PREVIEW_BASE_DOMAIN`을 조합해 접속 주소를 자동 생성하고, Ingress와 PR 댓글 URL에 사용했습니다.

### 03-2 SSM 기반 비공개 클러스터 배포

GitHub-hosted runner에서 외부에 공개하지 않은 K3s 클러스터에 배포해야 했습니다. API 서버를 노출하는 대신 AWS SSM SendCommand로 K3s 서버에 배포 명령을 전달했습니다. Runner에는 Kubernetes 자격증명을 전달하지 않고, 리소스 생성·갱신·삭제는 K3s 서버 내부에서 수행했습니다.

### 03-3 정리 경로 이중화

PR Close 이벤트를 처리하는 Workflow가 실패하거나 이벤트가 누락되면 Preview 환경이 남을 수 있어, 정리 경로를 두 가지로 나눴습니다. PR Close 시 DB cleanup Job을 실행한 뒤 namespace를 삭제하고, 누락된 정리는 TTL 기반 CronJob으로 처리했습니다.

CronJob은 5분마다 `preview-environment=true` namespace를 조회해, `expires-at` annotation에 기록된 만료 시각이 지난 환경의 DB와 namespace를 정리하도록 했습니다.

### 03-4 실행 결과

- PR 생성·업데이트 시 Preview 환경 자동 생성·갱신, PR 댓글에 URL과 만료 시각 기록
- 여러 PR의 Namespace·DB·Host를 각각 분리해 동시에 운영
- PR Close 시 전용 DB·Namespace 즉시 정리, 이벤트 누락 시 TTL 후속 정리
- K3s API 외부 노출 없이 배포·삭제 완료

# 04 AWS 인프라 검증·장애·부하 테스트

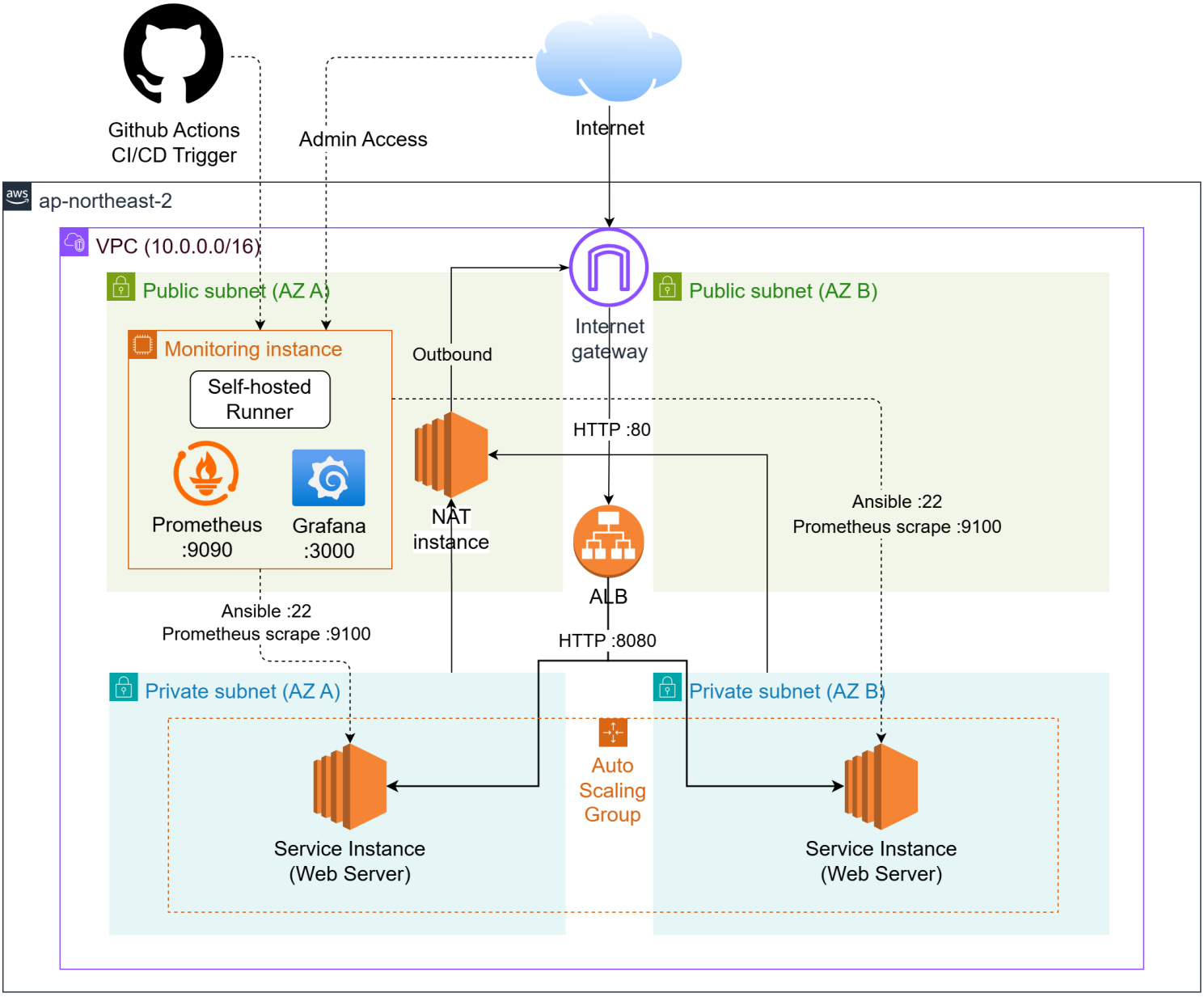
팀 프로젝트 · IaC 기반 클라우드 인프라 자동화

**2026.01.26 – 03.04 · 5명**

https://github.com/z-kim/ktcloudmini-aws-infra-tests

팀에서 Terraform·Ansible·GitHub Actions로 구성한 AWS 웹 인프라를 대상으로 장애 복구·오토스케일링 및 부하 상황에서의 동작을 검증했습니다.

저는 pytest·boto3로 인프라 상태를 확인하고, Locust로 외부 요청의 성공률과 응답 성능을 측정하는 테스트를 구현했습니다.




검증 대상이 된 팀 AWS 인프라 — ALB · ASG · EC2 · Prometheus

## 04-1 검증 로직과 대상 인프라 분리

ALB·ASG·Target Group 정보는 테스트 코드에 고정하지 않고 Terraform Output 또는 환경 변수에서 읽도록 구성했습니다. 같은 ALB·ASG 구조와 테스트에 필요한 엔드포인트(`/`, `/health`, `/kill`, `/work`)를 갖춘 AWS 환경에서는 `ALB_URL`, `ASG_NAME`, `TG_ARN`과 목표 임계치만 변경해 검증 코드를 재사용할 수 있도록 했습니다.

개발 단계에서는 팀 인프라와 별개로 최소 AWS 환경(`fixtures/`)을 Terraform으로 구성해 장애·복구·스케일링 시나리오를 반복 검증했습니다. 팀 인프라 완성 후에는 대상 정보만 교체해 동일한 검증 코드를 적용했습니다.

## 04-2 인프라 상태와 클라이언트 요청을 분리해 검증

리소스 상태만으로는 실제 서비스 가용성을 판단하기 어려워, 인프라 동작과 외부 요청 결과를 나눠 확인했습니다. pytest·boto3로 Target Group의 Healthy Target 수, AZ 분산, 인스턴스 교체와 스케일 동작을 확인했습니다. Locust로는 외부 요청의 성공 여부와 응답 시간을 측정했습니다.

Locust 시나리오는 역할이 다른 세 종류의 가상 사용자로 구성했습니다.

- **ObserveUser** — `/` 요청으로 응답 시간, 성공 여부, 처리 인스턴스 분포를 측정
- **ScalingUser** — `/work` 호출로 CPU 부하를 발생시켜 Scale-out 유도
- **FaultUser** — 장애 주입 전용 사용자를 1명으로 제한해 `/kill` 호출

부하·장애 주입 요청이 성능 지표에 섞이지 않도록, 서비스 가용성·응답 성능 지표에는 관측 요청(OBSERVE)만 반영했습니다.

## 04-3 장애·부하 테스트 결과

계단식 부하 테스트에서는 사용자 수를 단계적으로 늘리며 각 단계의 관측 요청 P95를 측정했습니다. P95 1,000ms 초과를 종료 기준으로 정하고, 해당 단계의 사용자 수를 기록했습니다.

| 시나리오 | 결과 |
|---|---|
| 정상 상태 | 29,093 req · 실패 0 · P95 13ms · 두 인스턴스 50:50 분산 |
| 단일 앱 장애 | 약 40초간 사용자 체감 장애 · 신규 인스턴스 참여까지 3분 46초 |
| 전체 앱 장애 | 완전 서비스 중단 약 232초 |
| Scale-out / in | 관측 요청 112,445건 · 실패 0 · P95 16ms · 요청 중단 없음 |
| 계단식 부하 | 20단계(400명)에서 P95 1,113ms · 종료 기준 초과 |

## 04-4 검증에서 확인한 점과 개선

**서비스 응답 회복과 인스턴스 교체 시점.** 애플리케이션 장애 후 ALB가 장애 Target을 제외한 뒤에는 외부 요청에서 HTTP 500 응답이 더 이상 관측되지 않았습니다. ASG가 새로 투입한 인스턴스는 그보다 늦게 요청 처리를 시작했습니다. 이를 통해 장애 복구를 평가할 때는 서비스 응답이 정상화된 시점과 신규 인스턴스가 요청 처리에 참여한 시점을 구분해야 한다는 점을 알게 되었습니다.

**Scale-in 미동작 원인과 해결.** 초기 Scaling 테스트에서 부하를 중단해도 Scale-in이 발생하지 않았습니다. CloudWatch Alarm을 확인해 missing data 처리 설정이 원인임을 파악했고, 담당자에게 설정 수정을 요청한 뒤 정상 동작을 확인했습니다.

**서버 측 지표와 외부 요청 성능의 차이.** 부하가 증가하는 동안 Grafana에서 확인한 서버 측 응답 시간은 거의 변하지 않았지만, 외부에서 측정한 P95는 크게 증가했습니다. 서비스 성능을 평가할 때는 서버 측 지표와 외부 요청의 응답 시간·성공률을 함께 확인해야 한다고 판단하게 되었습니다.

## 프로젝트 간 연결

### AWS 인프라 검증·장애·부하 테스트 → 멀티클라우드 DR 운영·검증 자동화

- AWS 인프라 검증에서 리소스 상태와 실제 요청 결과가 다를 수 있음을 확인했습니다. DR 프로젝트에서는 이 경험을 바탕으로 Kubernetes 상태나 Workflow 완료 여부뿐 아니라, 실제 주문 수신 재개와 Accepted 주문의 최종 DB 반영까지 검증 기준에 포함했습니다.

### K3s·Argo CD GitOps 운영 구성 → PR Preview 환경 생성·정리 자동화

- K3s에서 워크로드를 배포하고 운영 환경을 구성한 경험을 바탕으로, PR마다 Kubernetes Namespace와 전용 DB를 생성·정리하는 과정을 GitHub Actions로 자동화했습니다.

### K3s·Argo CD GitOps 운영 구성 → 멀티클라우드 DR 운영·검증 자동화

- K3s에서 사용한 Argo CD App-of-Apps 구조와 Sync Wave 기반 배포 순서 관리, base·overlay 방식을 DR 프로젝트에도 적용했습니다. DR에서는 EKS·AKS와 운영 상태에 맞게 구성을 분리하고, External Secrets·HPA도 각 환경에 맞춰 적용했습니다.
- GitHub App은 K3s·GitOps 프로젝트의 Private Repository 인증에서 도입했고, DR 프로젝트에서는 GitOps 저장소 변경·DR 상태 기록, 애플리케이션 저장소 조회, DR Workflow 실행 요청으로 용도를 나눠 각 작업에 필요한 최소 권한만 부여했습니다.

**읽어주셔서 감사합니다**

김지효

z33hyo@gmail.com · github.com/z-kim